

MANUAL TÉCNICO – Proyecto Sancarlista Shop

1. Descripción general

Sancarlista Shop es un sistema de gestión de ventas que simula el funcionamiento de una tienda en línea.

Permite la interacción entre tres tipos de usuarios: **Administrador**, **Vendedor** y **Cliente**.

Cada usuario tiene una interfaz gráfica distinta construida en **Java Swing**, y la persistencia de datos se maneja mediante **serialización de objetos (.ser)**.

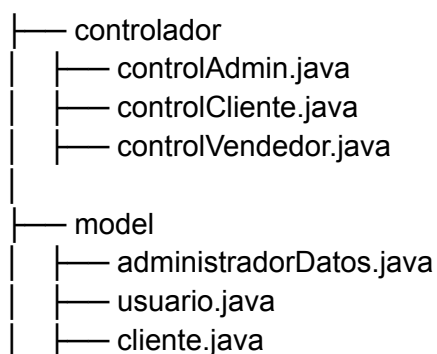
Los reportes se generan en formato **PDF con iText 5**.

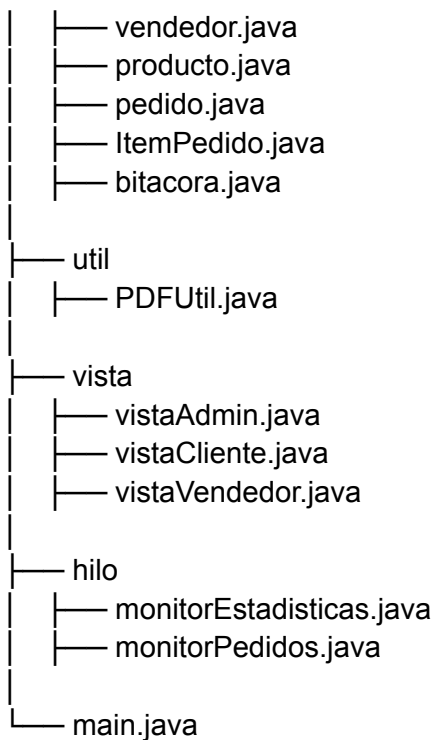
2. Requerimientos técnicos

Tipo	Detalle
Lenguaje	Java 8 o superior
IDE recomendado	NetBeans o IntelliJ IDEA
Librerías externas	iTextpdf-5.x.jar (para generación de reportes PDF)
Interfaz gráfica	Java Swing
Persistencia de datos	Archivos serializados .ser
Patrón de diseño	MVC (Modelo – Vista – Controlador)

3. Arquitectura del sistema

El sistema se estructura en paquetes:





4. Descripción de clases principales

administradorDatos

```
1 package model;
2
3 import java.io.Serializable;
4
5 public class administradorDatos implements Serializable {
6     private usuario[] usuarios;
7     private producto[] productos;
8     private pedido[] historialGlobal;
9     private bitacora[] bitacoras;
10    private stockMovimiento[] movimientos;
11
12    public administradorDatos() {
13        usuarios = new usuario[0];
14        productos = new producto[0];
15        historialGlobal = new pedido[0];
16        bitacoras = new bitacora[0];
17        movimientos = new stockMovimiento[0];
18    }
19
20    // --- getters (devuelven copias para seguridad) ---
21    public usuario[] getAllUsuarios() { return copiarUsuarios(usuarios); }
22    public producto[] getProductos() { return copiarProductos(productos); }
23    public pedido[] getHistorialGlobal() { return copiarPedidos(historialGlobal); }
24    public bitacora[] getBitacoras() { return copiarBitacoras(bitacoras); }
25    public stockMovimiento[] getMovimientos() { return copiarMovimientos(movimientos); }
26
27    // --- adders (expandir arreglos) ---
28    public void agregarUsuario(usuario u) { usuarios = pushUsuario(usuarios, u); }
29    public void agregarProducto(producto p) { productos = pushProducto(productos, p); }
30    public void agregarPedido(pedido p) { historialGlobal = pushPedido(historialGlobal, p); }
31    public void agregarBitacora(bitacora b) { bitacoras = pushBitacora(bitacoras, b); }
32    public void agregarMovimiento(stockMovimiento m) { movimientos = pushMovimiento(movimientos, m); }
```

producto

```
1 package model;
2
3 import java.io.Serializable;
4 import java.text.SimpleDateFormat;
5 import java.util.Date;
6
7 public class producto implements Serializable {
8     private String codigo;
9     private String nombre;
10    private String categoria;
11    private double precio;
12    private int stock;
13    private String atributoEspecifico; // fecha/material/meses etc.
14    private String lastUpdate; // NUEVO: última actualización de stock
15
16    public producto(String codigo, String nombre, String categoria, double precio) {
17        this.codigo = codigo;
18        this.nombre = nombre;
19        this.categoria = categoria;
20        this.precio = precio;
21        this.stock = 0;
22        this.atributoEspecifico = "";
23        touchUpdate();
24    }
25
26    public String getCodigo() { return codigo; }
27    public String getNombre() { return nombre; }
28    public String getCategoria() { return categoria; }
29    public double getPrecio() { return precio; }
30    public int getStock() { return stock; }
31    public String getAtributoEspecifico() { return atributoEspecifico == null ? "" : atributoEspecifico; }
32    public String getLastUpdate() { return lastUpdate; }
```

pedido

```
1 package model;
2
3 import java.io.Serializable;
4 import java.util.Date;
5
6 public class pedido implements Serializable {
7     private String codigoCliente;
8     private String codigoVendedor;
9     private itemPedido[] items;
10    private String estado;
11    private Date fecha;
12
13    // Constructor principal
14    public pedido(String codigoCliente, itemPedido[] items) {
15        this.codigoCliente = codigoCliente;
16        this.codigoVendedor = ""; // se asigna al confirmar el pedido
17        this.items = items;
18        this.estado = "Pendiente";
19        this.fecha = new Date();
20    }
21
22    // Constructor vacío (para uso interno o serialización)
23    public pedido() {
24        this.codigoCliente = "";
25        this.codigoVendedor = "";
26        this.items = new itemPedido[0];
27        this.estado = "Pendiente";
28        this.fecha = new Date();
29    }
30
31    // ----- GETTERS -----
32    public String getCodigoCliente() {
```

controlAdmin

```
1 package controlador;
2
3 import model.*;
4 import util.PDFUtil;
5 import javax.swing.*;
6 import java.io.FileWriter;
7 import java.util.List;
8
9 public class controlAdmin {
10     private administradorDatos data;
11     private administrador admin;
12
13     public controlAdmin(administradorDatos data, administrador admin) {
14         this.data = data;
15         this.admin = admin;
16     }
17
18     public bitacora[] getBitacoras() {
19         return data.getBitacoras();
20     }
21
22
23     public void exportarBitacoraCSV() {
24         try (FileWriter fw = new FileWriter("bitacora.csv")) {
25             for (bitacora b : data.getBitacoras()) {
26                 fw.write(b.getFecha() + "," + b.getUsuario() + "," + b.getOperacion() + "," + b.getDescripcion() + "\n");
27             }
28             JOptionPane.showMessageDialog(null, "Bitácora exportada correctamente.");
29         } catch (Exception e) {
30             JOptionPane.showMessageDialog(null, "Error al exportar la bitácora: " + e.getMessage());
31         }
32     }
```

controlVendedor

```
1 package controlador;
2
3 import model.*;
4 import javax.swing.*;
5 import java.io.*;
6
7 public class controlVendedor {
8     private administradorDatos data;
9     private vendedor user;
10
11     public controlVendedor(administradorDatos data, vendedor user) {
12         this.data = data; this.user = user;
13     }
14
15     public administradorDatos getData() { return data; }
16
17     public producto[] getProductos() { return data.getProductos(); }
18
19     public pedido[] getPedidosPendientes() {
20         // contar
21         pedido[] all = data.getHistorialGlobal();
22         int c = 0;
23         for (pedido p : all) if ("Pendiente".equals(p.getEstado())) c++;
24         // crear
25         pedido[] pend = new pedido[c];
26         int j = 0;
27         for (pedido p : all) if ("Pendiente".equals(p.getEstado())) pend[j++] = p;
28         return pend;
29     }
30
31     public void confirmarPedido(pedido p) {
32         p.confirmar(user.getCodigo());
33     }
```

controlCliente

```
1 package controlador;
2
3 import model.*;
4 import javax.swing.*;
5
6 public class controlCliente {
7     private administradorDatos data;
8     private cliente user;
9
10    public controlCliente(administradorDatos data, cliente user) {
11        this.data = data; this.user = user;
12    }
13
14    public administradorDatos getData() { return data; }
15
16    public producto[] getProductosDisponibles() { return data.getProductos(); }
17
18    public itemPedido[] getCarritoItems() { return user.getCarrito(); }
19
20    public void agregarAlCarrito(String codigo, int cantidad) {
21        producto p = data.buscarProducto(codigo);
22        if (p == null) { JOptionPane.showMessageDialog(null, "Producto no encontrado."); return; }
23        if (cantidad <= 0) { JOptionPane.showMessageDialog(null, "Cantidad > 0"); return; }
24        if (cantidad > p.getStock()) {
25            JOptionPane.showMessageDialog(null, "Stock insuficiente para " + p.getNombre() + " (Stock: " + p.getStock() + ")");
26            return;
27        }
28        user.agregarItem(codigo, cantidad);
29        data.agregarBitacora(new bitacora(user.getCodigo(), "Agregar al carrito", "Agregó " + cantidad + " de " + p.getNombre()));
30    }
31
32    public void eliminarDelCarrito(int index) {
```

PDFUtil

```
1 package util;
2
3 import com.itextpdf.text.*;
4 import com.itextpdf.text.pdf.*;
5 import java.io.FileOutputStream;
6 import java.text.SimpleDateFormat;
7 import java.util.*;
8 import model.*;
9
10 public class PDFUtil {
11
12    public static void generarReporte(String tipo, administradorDatos data) {
13        String fileName = new SimpleDateFormat("dd_MM_yyyy_HH_mm_ss").format(new Date()) + "_" + tipo + ".pdf";
14        try {
15            Document doc = new Document(PageSize.A4);
16            PdfWriter.getInstance(doc, new FileOutputStream(fileName));
17            doc.open();
18
19            // Encabezado del documento
20            doc.addTitle("Reporte: " + tipo);
21            doc.addAuthor("Sancarlista Shop");
22
23            Paragraph titulo = new Paragraph("REPORTE: " + tipo.toUpperCase(),
24                FontFactory.getFont(FontFactory.HELVETICA_BOLD, 16));
25            titulo.setAlignment(Element.ALIGN_CENTER);
26            doc.add(titulo);
27            doc.add(new Paragraph("Generado: " + new SimpleDateFormat("dd/MM/yyyy HH:mm:ss").format(new Date())));
28            doc.add(new Paragraph(" "));
29            doc.add(new Paragraph(" "));
30
31            // Selección del tipo de reporte
```

bitacora

```
1  package model;
2
3  import java.io.Serializable;
4  import java.text.SimpleDateFormat;
5  import java.util.Date;
6
7  public class bitacora implements Serializable {
8      private String usuario;
9      private String operacion;
10     private String descripcion;
11     private String fecha;
12
13     public bitacora(String usuario, String operacion, String descripcion) {
14         this.usuario = usuario;
15         this.operacion = operacion;
16         this.descripcion = descripcion;
17         this.fecha = new SimpleDateFormat("dd/MM/yyyy HH:mm:ss").format(new Date());
18     }
19
20     public String getUsuario() { return usuario; }
21     public String getOperacion() { return operacion; }
22     public String getDescripcion() { return descripcion; }
23     public String getFecha() { return fecha; }
24
25     @Override
26     public String toString() {
27         return "[" + fecha + "] " + usuario + " - " + operacion + ": " + descripcion;
28     }
29 }
30
```

5. Lógica de funcionamiento

Inicio del programa

- Se cargan los datos desde `data.ser`.
- Se muestran las credenciales de acceso.

Login

- Según el tipo de usuario, se abre la vista correspondiente.

Operaciones principales

- Cliente: compra productos, genera pedidos.
- Vendedor: gestiona stock y confirma pedidos.
- Administrador: genera reportes y revisa bitácoras.

Persistencia

- Al cerrar el sistema, se guardan los objetos serializados.

6. Hilos (paquete `hilo`)

Los monitores (`monitorPedidos`, `monitorEstadisticas`) ejecutan tareas en segundo plano:

- Actualización periódica de estadísticas.
- Verificación de pedidos pendientes y confirmados.
- Registro en bitácora automática.

7. Requerimientos de instalación

Recurso	Descripción
Java SDK	8 o superior
Librería externa	<code>iTextpdf-5.5.13.jar</code>
IDE	NetBeans, Eclipse o IntelliJ
Sistema Operativo	Windows / Linux / macOS
Archivo de datos	<code>data.ser</code> autogenerado

8. Pruebas básicas

Prueba	Resultado esperado
Crear producto nuevo	Aparece en catálogo y se guarda
Confirmar pedido	Cambia estado a “Confirmado”
Generar reporte PDF	Archivo <code>.pdf</code> guardado correctamente
Cierre y reapertura del sistema	Datos cargados desde <code>data.ser</code>

9. Control de errores

- Manejo de archivos mediante `try-catch` para evitar interrupciones.
- Validación de entradas vacías o nulas.
- Confirmaciones con `JOptionPane`.
- Prevención de ventas con stock insuficiente.